

Übung 6.87 Im Internet geht das Gerücht um, dass George Lucas (der Schöpfer der Star-Wars-Filme) eine Formel benutzt, um Namen für die Charaktere in seinen Geschichten zu erzeugen (Jar Jar Binks, Obiwan Kenobi etc.). Diese Formel lautet – angeblich:

Ihr Star-Wars-Vorname:

- 1** Nehmen Sie die ersten drei Buchstaben Ihres Nachnamens.
- 2** Fügen Sie diesen die ersten beiden Buchstaben Ihres Vornamens hinzu.

Ihr Star-Wars-Nachname:

- 1** Nehmen Sie die ersten beiden Buchstaben vom Mädchennamen Ihrer Mutter.
- 2** Fügen Sie diesen die ersten drei Buchstaben Ihrer Geburtsstadt hinzu.

Ihre Aufgabe lautet nun: Erzeugen Sie ein BlueJ-Projekt namens *Star-Wars*. Legen Sie darin eine Klasse **Namengenerator** an. Diese Klasse soll eine Methode **generiereStarWarsNamen** haben, die einen Namen nach dem oben angegebenen Schema produziert. Sie werden dazu die Methode **substring** der Klasse **String** benötigen, mit der Teile aus einer Zeichenkette herausgeschnitten werden können.

Übung 6.88 Die folgende Methode versucht, einen String in Großbuchstaben auszugeben:

```
public void inGrossbuchstabenAusgeben(String s)
{
    s.toUpperCase(); System.out.println(s);
}
```

Diese Methode funktioniert jedoch nicht. Finden Sie heraus, warum nicht, und erklären Sie es. Wie sollte es richtig aussehen?

Übung 6.89 Nehmen Sie an, wir wollen die Werte zweier **int**-Variablen **a** und **b** vertauschen. Dazu schreiben wir die Methode

```
public void tausche(int i1, int i2)
{
    int tmp = i1;
    i1 = i2;
    i2 = tmp;
}
```

Anschließend rufen Sie diese Methode mit unseren Variablen **a** und **b** auf:

```
tausche(a,b);
```

Sind **a** und **b** nach diesem Aufruf vertauscht? Wenn Sie es testen, werden Sie sehen, dass sie es nicht sind! Warum funktioniert dies nicht? Erklären Sie es detailliert.

KAPITEL

7

Sammlungen mit fester Größe – Arrays



Lernziele

Zentrale Konzepte in diesem Kapitel: Arrays, `for`-Schleife

Java-Konstrukte in diesem Kapitel: Array, `for`-Schleife, `++`, Konditionaloperator, `?:`

Der Hauptfokus dieses Kapitels ist die Verwendung von Array-Objekten für Sammlungen fester Größe und die `for`-Schleife.

7.1 Sammlungen fester Größe

In Kapitel 4 haben wir uns angesehen, in welchen Bibliotheksklassen, zum Beispiel **ArrayList**, wir Sammlungen von Objekten verwalten können. Wir sind in Kapitel 6 auch weiteren Sammlungstypen wie **HashSet** und **HashMap** begegnet. Ein wesentliches Merkmal all dieser Sammlungstypen ist ihre flexible Kapazität – wir müssen nie festlegen, wie viele Elemente eine konkrete Sammlung aufnehmen wird, und die Anzahl der gespeicherten Elemente kann während der Lebensdauer der Sammlung beliebig variieren.

In diesem Kapitel geht es nun um Sammlungen, die keine flexible, sondern eine feste Kapazität besitzen – zu dem Zeitpunkt, an dem das Sammlungsobjekt erzeugt wird, müssen wir die maximale Anzahl der Elemente angeben, die gespeichert werden können, und diese Anzahl kann auch nicht mehr geändert werden. Das mag zunächst wie eine unnötige Einschränkung erscheinen. Es gibt allerdings Situationen, in denen wir im Voraus wissen, wie viele Elemente wir in einer Sammlung ablegen wollen, und diese Anzahl bleibt dann über die Lebenszeit der Sammlung konstant. Für solche Situationen haben wir die Möglichkeit, eine Sammlung mit einer festen Größe zum Speichern der Objekte zu wählen.

7.2 Arrays

Eine Sammlung mit einer festen Größe wird *Array* genannt. Obwohl die feste Größe bei Arrays in vielen Situationen ein deutlicher Nachteil sein kann, haben sie gegenüber Sammlungen mit flexibler Größe mindestens zwei Vorteile:

- Der Zugriff auf die Elemente in einem Array ist häufig sehr viel effizienter als auf die Elemente einer vergleichbaren Sammlung mit flexibler Größe.

Konzept

Ein **Array** ist eine besondere Art von Sammlung, die eine festgelegte Anzahl von Elementen halten kann.

- In Arrays können neben Objekten auch Werte der primitiven Typen abgelegt werden. Sammlungen mit flexibler Größe können ausschließlich Objekte speichern.¹

Ein weiterer Unterschied ist, dass es für Arrays eine spezielle syntaktische Unterstützung in Java gibt – der Zugriff auf ihre Elemente erfolgt mit einer besonderen Syntax, die von den üblichen Methodenaufrufen abweicht. Dies hat in erster Linie historische Gründe: Arrays sind die älteste Sammlungsstruktur in Programmiersprachen, die Syntax für den Umgang mit Arrays hat sich über Jahrzehnte hinweg entwickelt. In Java wurde die gleiche Syntax beibehalten, die sich in anderen Sprachen bereits durchgesetzt hatte. Dadurch sollte ein Umstieg für Programmierer anderer Sprachen erleichtert werden, obwohl diese Syntax nicht konsistent mit dem Rest der Syntax von Java ist.

In den folgenden Abschnitten werden wir zeigen, wie Arrays benutzt werden können, um Sammlung mit fester Größe zu verwalten. Wir werden auch ein neues Schleifenkonstrukt vorstellen, das im Zusammenhang mit Arrays eine wichtige Rolle spielt – die **for**-Schleife. (Beachten Sie, dass die **for**-Schleife sich von der **for-each**-Schleife unterscheidet.)

7.3 Die Analyse einer Logdatei

Webserver verwalten üblicherweise einige Logdateien, in denen sie Informationen über die Zugriffe auf die Webseiten ablegen, die sie zur Verfügung stellen. Mit geeigneten Werkzeugen können die Betreiber der Webdienste daraus nützliche Informationen extrahieren, beispielsweise:

- welche die beliebteste Seite im Angebot ist
- über welche Sites die Benutzer auf die eigene Site geleitet wurden
- ob andere Webseiten defekte Links auf die eigenen Seiten haben
- wie viele Daten an Kunden geliefert wurden
- zu welchen Zeiten über einen Tag, eine Woche oder einen Monat die Zugriffe besonders zahlreich sind

Solche Informationen können den Administratoren bei der Entscheidung helfen, ob beispielsweise leistungsfähigere Server angeschafft werden müssen oder welche Zeiten aufgrund der geringen Zugriffe am besten für Wartungsarbeiten geeignet sind.

Das Projekt *Weblog-Auswertung* enthält eine Anwendung, die eine Analyse der Daten eines Webserver durchführt. Der Server schreibt bei jedem Zugriff auf eine Seite eine Zeile in eine Logdatei. Ein Beispiel für eine solche Logdatei mit dem Namen *weblog.txt* ist im Projektordner enthalten. Jede Zeile enthält das Datum und die Uhrzeit eines Zugriffs in folgendem Format:

Jahr Monat Tag Stunde Minute

Die folgende Zeile beispielsweise protokolliert einen Zugriff um 03:45 Uhr am 7. Juni 2017:

2017 06 07 03 45

¹ Auch wenn die Autoboxing-Funktion, die wir in Kapitel 6 beschrieben haben, einen Mechanismus bietet, mit dem wir auch primitive Typen in flexible Sammlungen einfügen, können dennoch nur Arrays sie direkt speichern.

Das Projekt besteht aus fünf Klassen: **ProtokollAuswerter**, **LogdateiLeser**, **Logeintrag**, **LogzeilenZerleger** und **LogdateiErzeuger**. Wir werden überwiegend die Klasse **ProtokollAuswerter** betrachten, da diese Beispiele für die Erzeugung und Benutzung eines Arrays enthält (Listing 7.1). Spätere Übungen werden Sie dazu ermuntern, auch die Klasse **Logeintrag** zu untersuchen und zu modifizieren, da diese ebenfalls ein Array benutzt. Die Klassen **LogdateiLeser** und **LogzeilenZerleger** benutzen Konzepte der Sprache Java, die wir bisher noch nicht betrachtet haben, wir werden sie deshalb nicht im Detail untersuchen. Die Klasse **LogdateiErzeuger** erlaubt es Ihnen, eigene Logdateien mit Zufallsdaten zu erzeugen.

```
/**
 * Eine Klasse, die das Protokoll eines Webservers
 * in Hinsicht auf Zugriffe pro Stunde auswertet.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class ProtokollAuswerter
{
    // Hier werden die Zugriffe für die Stunden berechnet.
    private int[] zugriffeInStunde;
    // Verwendung eines LogdateiLesers.
    private LogdateiLeser leser;

    /**
     * Erzeuge ein Objekt, das die Zugriffe in den
     * Stunden eines Tages zählt.
     */
    public ProtokollAuswerter()
    {
        // Das Array-Objekt erzeugen, das die Zugriffe
        // in den einzelnen Stunden eines Tages zählt.
        zugriffeInStunde = new int[24];
        // Den Leser für die Logdatei erzeugen.
        leser = new LogdateiLeser();
    }

    /**
     * Analysiere die in der Logdatei erfassten Zugriffsdaten.
     */
    public void analysiereStundendaten()
    {
        while(leser.hasNext()) {
            Logeintrag eintrag = leser.next();
            int stunde = eintrag.gibStunde();
            zugriffeInStunde[stunde]++;
        }
    }

    /**
     * Gib die Anzahl der Zugriffe in den Stunden eines
     * Tages nach Stunden sortiert auf der Konsole aus.
     * Diese Werte sollten zuerst mit einem Aufruf von
     * 'analysiereStundendaten' berechnet werden.
     */
    public void stundendatenAusgeben()
    {
        System.out.println("Stunde: Zugriffe");
        for(int stunde = 0; stunde < zugriffeInStunde.length; stunde++) {
            System.out.println(stunde + ": " + zugriffeInStunde[stunde]);
        }
    }

    /**
     * Gib die Zeilen der Logdatei auf der Konsole aus.
     */
    public void logdateiAusgeben()
    {
        leser.datenAusgeben();
    }
}
```

Listing 7.1

Die Auswertung einer Logdatei für Webzugriffe.

Die Auswertung benutzt momentan lediglich Teile der Informationen, die in einer Logzeile stehen. Die Auswertung liefert Informationen darüber, welche Stunden des Tages durchschnittlich die meisten und die wenigsten Zugriffe haben. Dies geschieht, indem für jeden Zeitabschnitt von einer Stunde die Zugriffe gezählt werden, die in der Logdatei vermerkt sind.

Übung 7.1 Erkunden Sie das Projekt *Weblog-Auswertung*, indem Sie ein **ProtokollAuswerter**-Objekt erzeugen und seine Methode **analysiereStundendaten** aufrufen. Rufen Sie anschließend die Methode **stundendatenAusgeben** auf, die das Ergebnis der Analyse ausgibt. Welche sind die aktivsten Stunden eines Tages?

In den nächsten Abschnitten werden wir untersuchen, wie diese Klasse ihre Aufgabe mithilfe eines Arrays erfüllt.

7.3.1 Array-Variablen deklarieren

Die Klasse **ProtokollAuswerter** enthält ein Datenfeld mit einem Array-Typ:

```
private int[] zugriffeInStunde;
```

Das Erkennungsmerkmal bei der Deklaration einer Array-Variablen sind die eckigen Klammern als Teil des Typnamens: **int[]**. Dies besagt hier, dass die Variable **zugriffeInStunde** den Typ *Array von Ganzzahlen* hat. Wir sagen, dass **int** der *Basistyp* dieses speziellen Arrays ist, was bedeutet, dass das Array-Objekt Werte vom Typ **int** speichert. Die Deklaration einer Array-Variablen sollte nicht mit der ähnlich aussehenden Deklaration einer einfachen Variablen verwechselt werden:

```
int stunde; // eine einfache Variable vom Typ int
int[] zugriffeInStunde; // eine Array-Variable vom Typ int
```

Die Variable **stunde** kann den Wert einer einzelnen ganzen Zahl speichern, während **zugriffeInStunde** ein Array-Objekt referenzieren wird, sobald dieses erzeugt wurde. Durch die Deklaration einer Array-Variablen wird noch kein Objekt erzeugt. Dies geschieht erst als getrennter Schritt durch eine **new**-Anweisung, wie bei anderen Objekten auch.

Wir müssen uns diese ungewöhnliche Syntax noch einmal genauer ansehen. Die Deklaration **int[]** würde in konventioneller Syntax vermutlich eher als **Array<int>** erscheinen. Das hat, wie bereits gesagt, eher historische als logische Gründe. Sie sollten sich angewöhnen, es auf die gleiche Weise zu lesen: als „Array von ints“.

Übung 7.2 Schreiben Sie eine Deklaration für eine Array-Variable **leute**, die auf ein Array von **Person**-Objekten verweisen kann.

Übung 7.3 Schreiben Sie eine Deklaration für eine Array-Variable **verfügbar**, die auf ein Array von **boolean**-Werten verweisen kann.

Übung 7.4 Untersuchen Sie den Quelltext der Klasse `ProtokollAuswerter` und finden Sie alle Stellen, an denen das Datenfeld `zugriffeInStunde` verwendet wird. Kümmern Sie sich an dieser Stelle noch nicht darum, wozu diese unterschiedlichen Benutzungen dienen, da wir dies in den nächsten Abschnitten noch erläutern werden. Beachten Sie, wie häufig ein Paar eckige Klammern verwendet wird.

Übung 7.5 Was ist falsch an den folgenden Array-Deklarationen? Korrigieren Sie.

```
[ ]int zaehler;  
boolean[5000] belegt;
```

7.3.2 Array-Objekte erzeugen

Der nächste Schritt ist nun, eine Array-Variable mit einem Array-Objekt zu verknüpfen.

Der Konstruktor der Klasse `ProtokollAuswerter` enthält eine Anweisung zur Erzeugung eines Arrays von ganzen Zahlen:

```
zugriffeInStunde = new int[24];
```

Auch hier fällt auf, dass die Syntax von der normalen Objekterzeugung abweicht. So gibt es beispielsweise keine runden Klammern um die Parameter des Konstruktors, da ein Array-Objekt keinen Konstruktor hat. Der Code erzeugt ein Array-Objekt, in dem 24 unterschiedliche `int`-Werte gespeichert werden können, und lässt die Array-Variable `zugriffeInStunde` darauf verweisen. Der Wert 24 gibt die Größe des Arrays an und ist kein Konstruktorparameter. Abbildung 7.1 veranschaulicht das Ergebnis dieser Zuweisung.

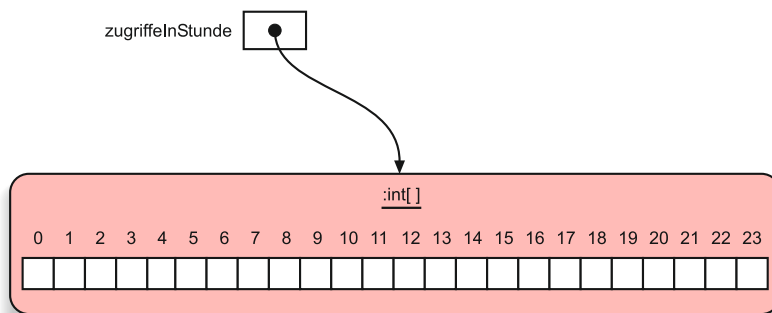


Abbildung 7.1

Ein Array für 24 `int`-Werte.

Die allgemeine Form für die Erzeugung eines Array-Objekts ist

```
new Typ[int-Ausdruck]
```

Die Auswahl des *Typs* legt fest, welchen Typ die Elemente des Arrays haben werden. Der *int-Ausdruck* legt die Größe des Arrays fest, also die feste Anzahl der Elemente, die in dem Array gespeichert werden können.

Wenn einer Array-Variablen ein Array-Objekt zugewiesen wird, dann muss der Typ dieses Objekts zum Typ der Variablen passen. Die Zuweisung an `zugriffeIn-`

Stunde ist zulässig, weil das Array-Objekt ein Array von ganzen Zahlen ist und **zugriffeInStunde** eine Variable für ein Array von ganzen Zahlen. Die folgende Zeile deklariert eine Variable für ein Array von Zeichenketten und lässt diese auf ein Array verweisen, das eine Kapazität für zehn Zeichenketten hat:

```
String[] namen = new String[10];
```

Es ist wichtig zu verstehen, dass durch die Erzeugung des Arrays, das der Variablen **namen** zugewiesen wird, nicht 10 Zeichenketten erzeugt werden. Es wird lediglich eine Sammlung mit fester Größe erzeugt, in der 10 Zeichenketten gespeichert werden können. Diese Zeichenketten werden möglicherweise an einer anderen Stelle in der Klasse erzeugt, zu der **namen** gehört. Unmittelbar nach seiner Erzeugung kann ein Array-Objekt als leer angesehen werden. Wenn es ein Array für Objekte ist, dann enthält das Array **null**-Werte für alle Elemente, und wenn es ein Array von ganzen Zahlen ist, dann sind alle Elemente auf null gesetzt. Im nächsten Abschnitt sehen wir uns an, wie Elemente in einem Array abgelegt (und wieder abgefragt) werden können.

Übung 7.6 Gegeben seien folgende Variablen-Deklarationen:

```
double[] temperaturen;  
String[] urls;  
Ticketautomat[] automaten;
```

Schreiben Sie Zuweisungen, die folgende Aufgaben ausführen: (a) die Variable **temperaturen** auf ein Array von 60 **double**-Werten verweisen lassen; (b) die Variable **urls** auf ein Array verweisen lassen, das 90 Zeichenketten speichern kann; (c) die Variable **automaten** auf ein Array verweisen lassen, das fünf **Ticketautomat**-Objekte halten kann.

Übung 7.7 Wie viele Zeichenketten werden durch die folgende Deklaration erzeugt?

```
String[] bezeichnungen = new String[20];
```

Übung 7.8 Was ist falsch an der folgenden Array-Erzeugung? Korrigieren Sie sie.

```
double[] preise = new double(50);
```

7.3.3 Array-Objekte benutzen

Auf die einzelnen Elemente eines Arrays wird über einen *Index* zugegriffen. Ein Index ist ein ganzzahliger Ausdruck zwischen eckigen Klammern, der auf den Namen der Variablen folgt. Beispielsweise:

```
bezeichnungen[6]  
automaten[0]  
leute[x + 10 - y]
```

Die Menge der gültigen Werte für einen Index-Ausdruck hängt von der Größe des Arrays ab, auf das dieser Index angewendet wird. Array-Indizes beginnen immer mit null und laufen bis zur Größe des Arrays minus eins. Die gültigen Indizes für das Array **zugriffeInStunde** sind somit die Werte von 0 bis inklusive 23.

Fallstrick

Zwei Fehler, die sehr häufig gemacht werden, sind zum einen die Annahme, dass die gültigen Indizes eines Arrays bei 1 beginnen, und zum anderen der Zugriff auf ein Element mit dem Index, der der Größe des Arrays entspricht. Die Benutzung von Indizes außerhalb der gültigen Grenzen eines Arrays führt zu einem Laufzeitfehler, der `ArrayIndexOutOfBoundsException` genannt wird.

Ausdrücke, die ein Element aus einem Array auswählen, können an allen Stellen stehen, an denen ein Wert des Array-Basistyps erwartet wird. Solche Ausdrücke können auf beiden Seiten einer Zuweisung verwendet werden. Hier sind einige Beispiele, in denen Array-Ausdrücke in verschiedenen Situationen verwendet werden:

```
bezeichnungen[5] = "Ende";
double haelfte = temperaturen[0] / 2;
System.out.println(leute[3].gibName());
automaten[0] = new Ticketautomat(500);
```

Die Benutzung eines Array-Index auf der linken Seite einer Zuweisung ist das Array-Gegenstück zum Aufruf einer verändernden Methode (oder `setze`-Methode), denn der Inhalt des Arrays wird dabei verändert. Die Benutzung an jeder anderen Stelle ist das Array-Gegenstück zu einer sondierenden Methode (oder `gib`-Methode).

7.3.4 Die Auswertung der Logdatei

Das Array `zugriffeInStunde`, das im Konstruktor der Klasse `ProtokollAuswerter` erzeugt wird, wird zum Speichern der Analysedaten der Zugriffswerte verwendet. Die Daten werden in der Methode `analysiereStundendaten` im Array abgelegt und in der Methode `stundendatenAusgeben` ausgelesen. Da die Aufgabe der `analysiere`-Methode ist, für jeden Stundenabschnitt die Anzahl der Zugriffe zu zählen, benötigt das Array 24 Plätze – einen für jeden Stundenabschnitt eines 24-Stunden-Tags. Der Auswerter delegiert das Einlesen der Logdatei an einen `LogdateiLeser`.

Die Klasse `LogdateiLeser` ist recht komplex, deshalb empfehlen wir, dass Sie nicht zu viel Zeit mit der Untersuchung ihrer Implementierung verbringen. Ihre Aufgabe ist das Aufteilen der Zeilen in einzelne Werte. Wir können von ihrer Implementierung abstrahieren, indem wir lediglich zwei ihrer Methoden betrachten:

```
public boolean hasNext()
public Logeintrag next()
```

Diese beiden Methoden entsprechen genau denen, die wir vom `Iterator`-Typ her kennen, und ein `LogdateiLeser` kann genauso verwendet werden, außer dass in diesem Fall nicht die Methode `remove` verwendet werden darf. Die Methode `hasNext` sagt dem Auswerter, ob mindestens ein weiterer Eintrag in der Logdatei existiert, und die Methode `next` liefert ein `LogEintrag`-Objekt, das die Werte der nächsten Zeile enthält.

Von jedem Logeintrag lässt sich die Methode `analysiereStundendaten` den Stundenwert geben:

```
int stunde = eintrag.gibStunde();
```


Wir wissen, dass der Wert der lokalen Variablen **stunde** immer im Bereich von 0 bis 23 liegt, und dies entspricht genau den gültigen Indizes im Array **zugriffeInStunde**. Jeder Eintrag in diesem Array wird als Zugriffszähler für den entsprechenden Stundenabschnitt benutzt. Wir wollen also jedes Mal, wenn ein Stundenwert gelesen wurde, den Zähler für die entsprechende Stunde erhöhen. Wir haben dies im Quelltext so formuliert:

```
zugriffeInStunde[stunde]++;
```

Beachten Sie, dass der im Array-Element gespeicherte Wert erhöht wird und nicht die Variable **stunde**. Die folgenden Alternativen sind beide äquivalent zu dieser Anweisung, da wir ein Array-Element genauso wie eine normale Variable verwenden können:

```
zugriffeInStunde[stunde] = zugriffeInStunde[stunde] + 1;
zugriffeInStunde[stunde] += 1;
```

Nach Ende der Ausführung der Methode **analysiereStundendaten** haben wir einen kompletten Satz von aufaddierten Zählern für jeden Stundenabschnitt des protokollierten Zeitraums.

Im nächsten Abschnitt sehen wir uns die Methode **stundendatenAusgeben** näher an, da diese eine neue Kontrollstruktur verwendet, die gut für das Iterieren über ein Array geeignet ist.

7.4 Die for-Schleife

Java definiert zwei Varianten von **for**-Schleifen, beide eingeleitet durch das Schlüsselwort *for*. In Abschnitt 4.9 haben wir die erste Variante, die *for-each-Schleife*, als ein geeignetes Mittel eingeführt, um über eine Sammlung mit flexibler Größe zu iterieren. Die zweite Variante, die *for-Schleife*, ist eine weitere iterative Kontrollstruktur,² die besonders geeignet ist, wenn

- wir einige Anweisungen wiederholen wollen und die Anzahl der Wiederholungen von vornherein feststeht;
- wir innerhalb der Schleife eine Variable benötigen, deren Wert sich bei jedem Durchlauf um einen festgelegten Wert – typischerweise um 1 – verändern soll.

Die **for**-Schleife eignet sich gut für Situationen, in denen eine *bestimmte Iteration* erforderlich ist. Beispielsweise benutzen wir eine **for**-Schleife häufig, wenn wir jedes Element eines Arrays bearbeiten wollen. Dies passt zu den Kriterien, denn die feststehende Häufigkeit korrespondiert mit der festen Größe des Arrays, und die Variable wird benötigt, um einen ansteigenden Index für das Array zu haben.

Eine **for**-Schleife hat die folgende allgemeine Form:

```
for(Initialisierung; Bedingung; Aktion nach dem Rumpf) {
    Anweisungen, die wiederholt werden sollen
}
```

² Es kommt vor, dass Personen, die den Unterschied zur **for-each**-Schleife verdeutlichen wollen, die **for**-Schleife als die „alte **for**-Schleife“ bezeichnen, weil sie von Anfang an in der Sprache Java definiert war. Die **for-each**-Schleife wird manchmal auch als „verbesserte **for**-Schleife“ (*enhanced for loop*) bezeichnet.

Das folgende konkrete Beispiel ist der Methode `stundendatenAusgeben` des Protokollauswerters entnommen:

```
for(int stunde = 0; stunde < zugriffeInStunde.length; stunde++) {
    System.out.println(stunde + ": " + zugriffeInStunde[stunde]);
}
```

Das Ergebnis dieser Anweisung ist, dass der Wert jedes Elements im Array ausgegeben wird, wobei die entsprechende Stundennummer vorangestellt wird. Beispielsweise:

```
0: 149
1: 149
2: 148
...
23: 166
```

Wenn wir diese **for**-Schleife mit der **for-each**-Schleife vergleichen, dann sehen wir den syntaktischen Unterschied in dem Bereich zwischen den runden Klammern im Schleifenkopf. In dieser **for**-Schleife stehen dort drei Abschnitte, die jeweils durch ein Semikolon voneinander getrennt sind.

Aus einer Designsicht auf die Programmiersprache wäre es schöner gewesen, wenn zwei verschiedene Schlüsselwörter für diese beiden Schleifenarten verwendet worden wären, etwa **for** und **foreach**. Dass für beide **for** verwendet wird, ist, wie gesagt, auch hier wieder historisch bedingt. Ältere Versionen der Sprache Java kannten die **for-each**-Schleife noch nicht. Als sie schließlich eingeführt wurde, wollten die Designer der Sprache nicht zusätzlich ein neues Schlüsselwort einführen, da dies Probleme mit bestehenden Programmen verursachen kann. Deshalb haben sie sich entschieden, das gleiche Schlüsselwort für beide Schleifen zu verwenden. Dies macht es uns etwas schwerer, diese beiden Schleifen voneinander zu unterscheiden, aber wir werden mit der Zeit die unterschiedlichen Kopfstrukturen auseinanderhalten können.

Auch wenn die **for**-Schleife oft für die bestimmte Iteration verwendet wird, bedeutet die Tatsache, dass sie von einem allgemeinen booleschen Ausdruck gesteuert wird, dass sie der **while**-Schleife näher steht als der **for-each**-Schleife. Wir können die Ausführung einer **for**-Schleife veranschaulichen, indem wir ihre allgemeine Form durch eine äquivalente **while**-Schleife formulieren:

```
Initialisierung
while(Bedingung) {
    Anweisungen, die wiederholt werden sollen
    Anweisung nach dem Rumpf
}
```

Entsprechend sähe der Rumpf von `stundendatenAusgeben` folgendermaßen aus:

```
int stunde = 0;
while(stunde < zugriffeInStunde.length) {
    System.out.println(stunde + ": " + zugriffeInStunde[stunde]);
    stunde++;
}
```

Aus dieser umgeschriebenen Version können wir ersehen, dass die „Anweisung nach dem Rumpf“ tatsächlich erst nach den Anweisungen im Schleifenrumpf

ausgeführt wird, obwohl diese Anweisung im Kopf der Schleife steht. Außerdem können wir sehen, dass die Initialisierung nur genau einmal ausgeführt wird – unmittelbar bevor die Bedingung das erste Mal geprüft wird.

In beiden Versionen ist die Anweisung

```
stunden < zugriffeInStunde.length
```

zu beachten. Sie zeigt zwei wichtige Aspekte:

- Alle Arrays enthalten ein Datenfeld **length**, das die festgelegte Größe eines Arrays hält. Der Wert dieses Datenfeldes wird immer dem Wert entsprechen, mit dem das Array-Objekt erzeugt wurde. In diesem Fall ist der Wert also 24.
- Die Bedingung benutzt den Kleiner-Operator **<**, um den Wert von **stunde** mit der Länge des Arrays zu vergleichen. In diesem Fall wird die Schleife also wiederholt, solange **stunde** niedriger ist als 24. Für den allgemeinen Fall, dass wir auf jedes Element in einem Array zugreifen wollen, hat der Kopf der **for**-Schleife folgende allgemeine Form:

```
for(int index = 0; index < array.length; index++)
```

Dies ist korrekt, denn wir wollen keinen Index verwenden, der gleich der Größe des Arrays ist – solch ein Element wird es bei einem Array niemals geben.

7.4.1 Arrays und die for-each-Schleife

Könnten wir die oben vorgestellte **for**-Schleife auch als **for-each**-Schleife schreiben? Die Antwort lautet: beinahe. Ein möglicher Ansatz für eine solche Schleife könnte beispielsweise folgendermaßen aussehen:

```
for(int wert : zugriffeInStunde) {
    System.out.println(": " + wert);
}
```

Dieser Code lässt sich übersetzen und ausführen (versuchen Sie es!). Wir können daran sehen, dass Arrays genauso wie Sammlungen in **for-each**-Schleifen verwendet werden können. In diesem konkreten Fall gibt es allerdings ein Problem: Die Stunde vor dem Doppelpunkt lässt sich auf diese Weise nicht so leicht ausgeben. Das Quelltextfragment verzichtet daher auf die Ausgabe der Stunde und gibt nur den Doppelpunkt und den Wert aus. Dies liegt schlicht daran, dass die **for-each**-Schleife keinen Schleifenzähler anbietet, der uns das Ausgeben der Stunde ermöglichen würde.

Um das Problem zu beheben, müssten wir unsere eigene Zählervariable definieren (ähnlich dem Beispiel mit der **while**-Schleife). Wir ziehen es aber vor, die traditionelle **for**-Schleife zu verwenden, die in diesem Fall prägnanter ist.

7.4.2 Die for-Schleife und Iteratoren

In Abschnitt 4.12.2 haben wir gesehen, dass wir zum Entfernen von Elementen aus einer Sammlung einen **Iterator** benötigen. Es gibt allerdings eine bestimmte Form der Verwendung einer **for**-Schleife mit einem **Iterator**, die für derlei Aufgaben ebenfalls geeignet ist. Angenommen wir möchten aus unserer Musiksammlung jeden Track entfernen, der von einem bestimmten Interpreten stammt. Dazu müssen wir

jeden Track in der Sammlung prüfen, was für den Einsatz einer **for-each**-Schleife spricht. Wie wir jedoch gelernt haben, ist dies hier nicht möglich. Stattdessen verwenden wir eine **for**-Schleife:

```
for(Iterator<Track> it = tracks.iterator(); it.hasNext(); ) {
    Track track = it.next();
    if (track.gibInterpret().equals(interpret)) {
        it.remove();
    }
}
```

Der springende Punkt hierbei ist, dass im Schleifenkopf kein „Aktion nach dem Rumpf“-Teil vorgesehen ist – sein Platz bleibt einfach leer. Dies ist erlaubt, wir müssen jedoch darauf achten, nach der Schleifenbedingung das Semikolon zu setzen. Die Verwendung einer **for**-Schleife anstelle einer **while**-Schleife macht klarer, dass wir jedes Element der Liste durchlaufen wollen.

Welche Schleife soll ich benutzen?

Wir haben drei verschiedene Schleifen kennengelernt: die **for**-Schleife, die **for-each**-Schleife und die **while**-Schleife. Wie Sie gesehen haben, können wir jede von ihnen benutzen, um ein bestimmtes Problem zu lösen. Normalerweise kann jede der Schleifen durch eine andere ersetzt werden. Für welche sollen Sie sich also jeweils entscheiden? Hier kommen ein paar Richtlinien:

- Wenn Sie alle Elemente in einer Sammlung bearbeiten wollen, dann ist die **for-each**-Schleife fast immer die eleganteste Lösung. Sie ist klar und knapp (bietet aber keinen Schleifenzähler).
- Wenn Sie eine Schleife benötigen, die sich nicht auf eine Sammlung bezieht (sondern irgendeine andere Aktion wiederholt ausführen soll), ist die **for-each**-Schleife nicht hilfreich. Sie müssen sich dann zwischen der **for**-Schleife und der **while**-Schleife entscheiden. Die **for-each**-Schleife ist ausschließlich für Sammlungen vorgesehen.
- Die **for**-Schleife ist gut geeignet, wenn Sie zu Beginn der Schleife wissen, wie viele Wiederholungen Sie benötigen. Diese Information kann in einer Variablen vorliegen, sollte aber während der Schleifenausführungen nicht geändert werden. Die **for**-Schleife ist auch gut geeignet, wenn Sie den Schleifenzähler explizit benötigen.
- Die **while**-Schleife sollte bevorzugt eingesetzt werden, wenn Sie zu Beginn der Schleife nicht wissen, wie oft die Schleife durchlaufen werden soll. Das Ende der Schleife kann während der Ausführung aufgrund einer Bedingung ermittelt werden (beispielsweise: eine Zeile aus einer Datei einlesen – wiederholt – bis wir das Ende der Datei erreichen).
- Wenn Sie während der Schleifendurchläufe Elemente aus einer Sammlung entfernen müssen, haben Sie zwei Alternativen: Wollen Sie die ganze Sammlung durchlaufen, sollten Sie eine **for**-Schleife zusammen mit einem **Iterator** wählen. Wenn Sie möglicherweise fertig werden, bevor Sie das Ende der Sammlung erreicht haben, können Sie auf eine **while**-Schleife zurückgreifen.

Übung 7.9 Überprüfen Sie, was passiert, wenn in der Bedingung der **for**-Schleife von **stundendatenAusgeben** fälschlich der Kleiner-gleich-Operator verwendet wird:

```
for(int stunde = 0; stunde <= zugriffeInStunde.length; stunde++)
```

Übung 7.10 Schreiben Sie den Rumpf von **stundendatenAusgeben** so um, dass die **for**-Schleife durch eine äquivalente **while**-Schleife ersetzt wird. Rufen Sie die geänderte Methode auf, um zu prüfen, ob sie noch dasselbe Resultat liefert.

Übung 7.11 Korrigieren Sie alle Fehler in der folgenden Methode.

```
/**
 * Gib alle Werte im Array noten aus, die höher als der
 * mittelwert sind.
 * @param noten ein Array mit Notenwerten.
 * @param mittelwert die Durchschnittsnote.
 */
public void hoehereAusgeben(double noten, double mittelwert)
{
    for(index = 0; index <= noten.length; index++) {
        if(noten[index] > mittelwert)
        { System.out.println(noten[index]);
        }
    }
}
```

Übung 7.12 Ändern Sie die Klasse **ProtokollAuswerter** so, dass ihr Konstruktor den Namen der zu analysierenden Protokolldatei übernehmen kann. Dieser Konstruktor soll den Dateinamen an den Konstruktor der Klasse **LogdateiLeser** übergeben. Erzeugen Sie anschließend mit der Klasse **LogdateiErzeuger** Ihre eigene Datei mit zufälligen Protokolleinträgen und analysieren Sie die Daten.

Übung 7.13 Vervollständigen Sie die folgende Methode **anzahlZugriffe**, die die Gesamtanzahl aller Zugriffe berechnet, die in der Logdatei protokolliert sind. Benutzen Sie dazu eine **for**-Schleife, die über **zugriffeInStunde** iteriert:

```
/**
 * Liefere die Anzahl aller Zugriffe, die in der Logdatei
 * protokolliert sind.
 */
public int anzahlZugriffe()
{
    int gesamt = 0;
    // Addiere den Wert jedes Elements in zugriffeInStunde
    // zu gesamt hinzu.
    ...
    return gesamt;
}
```

Übung 7.14 Fügen Sie eine Methode `anzahlZugriffe` in die Klasse `ProtokollAuswerter` ein und prüfen Sie, ob sie das richtige Ergebnis liefert.

Hinweis: Sie können die Prüfung vereinfachen, indem Sie den Auswerter eine Datei mit nur wenigen Zeilen lesen lassen. Auf diese Weise werden Sie leichter feststellen können, ob Ihre Methode das richtige Ergebnis liefert. Die Klasse `LogdateiLeser` hat einen Konstruktor mit dem folgenden Kopf, um aus einer bestimmten Datei lesen zu können:

```
/**
 * Erzeuge einen LogdateiLeser, der Einträge aus einer
 * Datei mit dem angegebenen Namen liest.
 * @param dateiname der Name der Logdatei.
 */
public LogdateiLeser(String dateiname)
```

Übung 7.15 Fügen Sie `ProtokollAuswerter` eine Methode `aktivsteStunde` hinzu, die die Stunde mit den meisten Zugriffen liefern soll. Sie können dies implementieren, indem Sie das Array `zugriffeInStunde` nach dem Element mit dem höchsten Wert durchsuchen. *Hinweis:* Müssen Sie jedes Element überprüfen, um den höchsten Wert zu finden? Falls ja, dann verwenden Sie eine `for`-Schleife oder eine `for-each`-Schleife. Welche ist besser geeignet für diesen Fall?

Übung 7.16 Fügen Sie zu `ProtokollAuswerter` eine Methode `ruhigsteStunde` hinzu, die die Stunde mit den wenigsten Zugriffen liefern soll. *Hinweis:* Dies klingt fast identisch zur vorigen Übung, aber hier gibt es eine kleine Falle für die Unaufmerksamen. Gehen Sie sicher, dass Ihre Methode mit Daten arbeitet, bei denen jede Stunde einen Wert größer als null hat.

Übung 7.17 Welche Stunde wird von Ihrer Methode `aktivsteStunde` zurückgeliefert, wenn mehr als eine Stunde den größten Wert hat?

Übung 7.18 Fügen Sie `ProtokollAuswerter` eine Methode hinzu, die den aktivsten zweistündigen Abschnitt findet. Lassen Sie die Methode den Wert der ersten Stunde dieses Abschnitts liefern.

Übung 7.19 *Zusatzaufgabe.* Speichern Sie das Projekt *Weblog-Auswertung* unter einem anderen Namen, damit Sie eine neue Version erstellen können, in der die gegebenen Daten ausführlicher ausgewertet werden. Zum Beispiel könnte die Information nützlich sein, welche Tage ruhiger sind als andere. Ist beispielsweise ein zyklisches Muster über die sieben Wochentage erkennbar? Um Auswertungen von Tages-, Monats- oder Jahresdaten durchführen zu können, müssen Sie einige Änderungen an der Klasse `LogEintrag` vornehmen. Diese speichert zwar alle Werte einer Logzeile, macht aber nur die Stunde und die Minute über sondierende Methoden zugänglich. Fügen Sie weitere Methoden hinzu, die auch die anderen Werte zugreifbar machen. Fügen Sie dann weitere Analysemethoden hinzu.

Übung 7.20 Zusatzaufgabe. Wenn Sie die vorige Aufgabe bearbeitet haben, können Sie das Format der Logdatei um weitere numerische Einträge erweitern. Beispielsweise speichern Webserver üblicherweise eine numerische Kennung, mit der der Zugriffserfolg protokolliert wird: Der Wert 200 steht für einen erfolgreichen Zugriff, 403 besagt, dass der Zugriff nicht erlaubt ist, und 404 bedeutet, dass die angeforderte Seite nicht gefunden werden konnte. Lassen Sie den Protokollauswerter Informationen über erfolgreiche und nicht erfolgreiche Zugriffe liefern. Diese Aufgabe ist sehr anspruchsvoll, da Sie jede Klasse des Projekts anpassen müssen.

Übung 7.21 Im Projekt *Laborkurse*, das wir in vorherigen Kapiteln untersucht haben, enthält die Klasse **Laborkurs** ein Datenfeld **studenten**, das eine Liste von Studenten verwaltet. Lesen Sie sich den Quelltext dieser Klasse durch, um die Konzepte wiederzuerkennen, die wir in diesem Kapitel vorgestellt haben.

Übung 7.22 Die Klasse **Laborkurs** schreibt eine Maximalzahl an Studenten vor, die sich in einen Kurs einschreiben können. Denken Sie, dass es unter diesem Gesichtspunkt angemessener wäre, statt der Sammlung mit flexibler Größe ein Array mit fester Größe für das Datenfeld **studenten** zu verwenden? Diskutieren Sie Vor- und Nachteile beider Alternativen.

Übung 7.23 Schreiben Sie die Methode **alleDateienAusgeben** in der Klasse **MusikSammlung** des Projekts *Musiksammlung-v3* so um, dass eine **for**-Schleife anstelle einer **for-each**-Schleife verwendet wird.

7.5 Das Projekt Automat

In den folgenden Abschnitten werden wir unsere Behandlung der Arrays ausweiten, um einige weitere typische Verwendungsmöglichkeiten zu zeigen. Wir werden zwei Projekte einsetzen, die auf *zellulären Automaten* basieren. Dies sind relativ einfache Rechenmodelle, die sowohl interessante als auch verblüffende „Verhaltensmuster“ aufweisen. Wir werden mit einigen eindimensionalen Beispielen anfangen und das Modell dann auf zwei Dimensionen erweitern, um den Einsatz von zweidimensionalen Arrays zu untersuchen.

Ein zellulärer Automat besteht aus einem Gitter von „Zellen“. Jede Zelle besitzt einen einfachen Status, der aus einem begrenzten Wertebereich besteht – oft einfach durch die Werte 0 und 1 dargestellt. Diese Werte könnten zum Beispiel als „an“ und „aus“ interpretiert werden als oder „tot“ und „lebend“ – aber die Bedeutung ist eigentlich beliebig. Wir könnten eine Zelle zwar als Klassendefinition implementieren, die ein einziges boolesches Feld sowie eine sondierende und eine verändernde Methode enthält, dies wäre aber vermutlich für einen einfachen Automaten eine Nummer zu groß. Deshalb repräsentieren wir den Automaten lieber als ein Array vom Typ **int[]**. Die Wahl von **int** anstelle von **boolean** wird es einfacher machen, die Veränderungen der Zustandswerte zu berechnen, die bei dieser speziellen Anwendung nötig sind.

In jedem Schritt des Modells wird für jede Zelle ein neuer Zustand basierend auf seinem vorhergehenden Zustand und den Zuständen seiner Nachbarn berechnet. Für einen eindimensionalen Automaten sind die Nachbarn die beiden Zellen links und rechts. Mit anderen Worten, für eine Zelle mit Index i sind die Nachbarn die Zellen mit den Indizes $(i-1)$ und $(i+1)$. Wir werden diese Zelle an Index i als die Zelle im „Zentrum“ bezeichnen. Automaten sind deshalb so interessant, weil die verschiedenartigen Regeln zum Berechnen neuer Zustände, basierend auf diesen drei Werten, nach mehreren Schritten ganz unterschiedliche Auswirkungen auf das Gesamtverhalten des Automaten haben.

Listing 7.2 zeigt einen ersten Versuch, einen Automaten aufgrund dieser Prinzipien zu implementieren. Sie finden die Klasse im Projekt *Automat-v1*. Die verwendete Regel zur Berechnung des neuen Zellzustands besteht darin, die Zustandswerte der beiden Nachbarn zum Zustandswert der Zelle zu addieren und auf diese Summe dann eine Modulo-2-Operation anzuwenden, sodass das Ergebnis entweder 0 oder 1 sein muss. Abbildung 7.2 zeigt das Ergebnis der wiederholten Anwendung dieser Aktualisierungsregel auf alle Zellen, wobei es im Ausgangszustand (Schritt 0) nur eine einzige Zelle mit Wert 1 gab und alle anderen Zellen mit 0 initialisiert waren. Nach dem ersten Schritt behält die ursprüngliche 1-Zelle den Wert 1, aber die beiden 0-Nachbarn erhalten jetzt auch den Wert 1, während alle anderen Zellen auf 0 bleiben. Die durch die folgenden Schritte hervorgerufenen Änderungen lassen ein symmetrisches Muster entstehen.

```
import java.util.Arrays;
```

```
/**
 * Modelliert einen eindimensionalen elementaren zellulären Automaten.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29 - Version 1
 */
public class Automat
{
    // Die Anzahl der Zellen.
    private final int anzahlZellen;
    // Der Zustand der Zellen.
    private int[] zustand;

    /**
     * Erzeuge einen eindimensionalen Automaten, der aus der angegebenen
     * Anzahl an Zellen besteht.
     * @param anzahlZellen die Anzahl der Zellen im Automaten
     */
    public Automat(int anzahlZellen)
    {
        this.anzahlZellen = anzahlZellen;
        zustand = new int[anzahlZellen];
        // den Automaten einrichten mit einer einzelnen 'an'-Zelle in der Mitte
        zustand[anzahlZellen / 2] = 1;
    }

    /**
     * Gib den aktuellen Zustand des Automaten aus.
     */
    public void ausgeben()
    {
        for(int zellenwert : zustand) {
            if(zellenwert == 1) {
                System.out.print("**");
            }
            else {
                System.out.print(" ");
            }
        }
        System.out.println();
    }
}
```

Listing 7.2

Die Klasse *Automat*.

```
/**
 * Aktualisiere den Automaten auf seinen nächsten Zustand.
 */
public void aktualisieren()
{
    // Baut den neuen Zustand in einem anderen Array auf.
    int[] naechsterZustand = new int[zustand.length];
    // Aktualisiert den Zustand jeder Zelle,
    // ausgehend von den Zuständen seiner beiden Nachbarn.
    for(int i = 0; i < zustand.length; i++) {
        int links, zentrum, rechts;
        if(i == 0) {
            links = 0;
        }
        else {
            links = zustand[i - 1];
        }
        zentrum = zustand[i];
        if(i + 1 < zustand.length) {
            rechts = zustand[i + 1];
        }
        else {
            rechts = 0;
        }
        naechsterZustand[i] = (links + zentrum + rechts) % 2;
    }
    zustand = naechsterZustand;
}

/**
 * Setze den Automaten zurück.
 */
public void zuruecksetzen()
{
    Arrays.fill(zustand, 0);
    // den Automaten einrichten mit einer einzelnen 'an'-Zelle.
    zustand[anzahlZellen / 2] = 1;
}
}
```

Abbildung 7.2

Die ersten Schritte des Automaten, der in *Automat-v1* implementiert ist.

Schritt	Zellzustände – leere Zellen sind im Zustand 0														
0							*								
1						*	*	*							
2					*		*		*						
3				*	*		*		*	*					
4				*			*				*				
5			*	*	*		*	*	*		*	*	*		
6		*		*			*				*		*		
7	*	*		*	*		*	*	*		*	*		*	*

- Übung 7.24 Öffnen Sie das Projekt *Automat-v1* und erzeugen Sie ein Objekt **AutomatController**. Es sollte eine Zeile mit einem einzelnen * auf dem Bildschirm ausgegeben werden, diese stellt den Anfangszustand des Automaten dar. Rufen Sie die Methode **schritt** ein paar Mal auf, um zu sehen, wie sich die Zustände entwickeln. Dann versuchen Sie die Methode **starten**.
- Übung 7.25 Nachdem Sie den Automaten ausgeführt haben, rufen Sie die Methode **zuruecksetzen** auf und wiederholen Sie den Prozess aus der vorigen Aufgabe. Ergeben sich genau dieselben Muster?

Übung 7.26 Wie viele Versionen der `fill`-Methode gibt es in der Klasse `java.util.Arrays`, die einen Parameter vom Typ `int[]` haben? Welchen Zweck haben diese Methoden? Wie wird eine davon in der `zuruecksetzen`-Methode von `Automat` eingesetzt?

Übung 7.27 Verändern Sie den Konstruktor von `Automat`, sodass am Anfang mehr als eine Zelle im 1-Zustand ist. Treten dieses Mal unterschiedliche Muster zutage? Sind diese Muster *deterministisch* – das heißt, führt ein bestimmter Startzustand des Automaten immer zum selben Muster?

7.5.1 Der Konditionaloperator

Während die Version des Automaten in *Automat-v1* funktioniert und interessante Resultate produziert, gibt es einige Verbesserungen, die wir an der Implementierung machen können. Wir beginnen, indem wir einen neuen Operator einführen, der häufig eingesetzt werden kann, um den Ausdruck von `if-else`-Anweisungen zu vereinfachen.

Sie haben vermutlich bemerkt, dass es in Listing 7.2 drei `if-else`-Anweisungen gibt, deren Struktur ungefähr so aussieht:

```
if(Bedingung) {
    tue etwas;
}
else {
    tue etwas Ähnliches;
}
```

In der Methode `ausgeben` haben wir zum Beispiel

```
if(zellenwert == 1) {
    System.out.print("*");
}
else {
    System.out.print(" ");
}
```

und in der Methode `aktualisieren` gibt es zwei `if-else`-Anweisungen, um den Variablen `links` und `rechts` Werte zuzuweisen, wobei sich die alternativen Aktionen nur in den Anweisungen auf der rechten Seite unterscheiden.

In solchen Situationen können wir häufig einen *Konditionaloperator* verwenden, der in Java aus den beiden Zeichen `?` und `:` besteht. Dieser Operator wird als *ternärer Operator* bezeichnet, weil er drei Operanden besitzt, im Gegensatz zu den bisher behandelten binären Operatoren wie `+` und `*`, die zwei Operanden haben. Dieser Operator wird eingesetzt, um einen von zwei alternativen Werten aufgrund der Auswertung eines booleschen Ausdrucks auszuwählen. Die allgemeine Form lautet:

```
Bedingung ? Wert1 : Wert2
```

Wenn die Bedingung **true** ist, dann wird der Wert des gesamten Ausdrucks *Wert1* sein, anderenfalls wird er *Wert2* sein. Zum Beispiel könnten wir den Rumpf der **for**-Schleife in der Methode **ausgeben** wie folgt umschreiben:

```
String wasAuszugebenIst = zellenWert == 1 ? "*" : " ";
System.out.print(wasAuszugebenIst);
```

oder noch kürzer:

```
System.out.print(zellenWert == 1 ? "*" : " ");
```

Übung 7.28 Schreiben Sie die beiden **if-else**-Anweisungen in der Schleife der Methode **aktualisieren** der **Automat**-Klasse von *Automat-v1* um, sodass für die Zuweisungen an **links** und **rechts** Konditionaloperatoren verwendet werden.

Übung 7.29 Warum haben wir in der **aktualisieren**-Methode ein neues Array, **naechsterZustand**, erzeugt, anstatt das Array **zustand** direkt zu verwenden? Verändern Sie die Zuweisung an **naechsterZustand** im Schleifenrumpf, sodass es nun eine Zuweisung an **zustand** wird, um festzustellen, ob es irgend-einen Unterschied im Verhalten des Automaten auslöst. Falls dies so ist, können Sie erklären, warum?

Übung 7.30 Können Sie eine Lösung für diese Probleme finden, die in der vorherigen Übung aufgezeigt wurden, ohne ein neues Array zu verwenden? *Hinweis:* Wie viele alte Werte der Zelle müssen bei jeder Iteration beibehalten werden? Welche Version ist Ihrer Meinung nach besser: ein vollständiges Array benutzen oder unsere Lösung? Begründen Sie Ihre Antwort.

7.5.2 Erste und letzte Iteration

Die bedeutendste Verbesserung, die wir an der **Automat**-Klasse in Listing 7.2 vornehmen können, ist die Struktur des Rumpfes der **for**-Schleife in der Methode **aktualisieren** zu überarbeiten. Im Moment ist der Code dort sehr seltsam, weil wir sehr vorsichtig beim Bearbeiten der ersten Zelle (**[0]**) und der letzten Zelle (**[anzahlZellen-1]**) sein müssen, um einen möglichen Zugriff auf nicht vorhandene Elemente des Arrays zu vermeiden (**[-1]** links von der ersten Zelle und **[anzahlZellen]** rechts von der letzten Zelle). Für diese nicht existierenden Elemente setzen wir einen Zustandswert von 0 ein, sodass wir bei der Berechnung des nächsten Zellzustands immer drei Werte haben.

Grundsätzlich gilt: Wenn innerhalb einer Schleife auf den Spezialfall der ersten oder letzten Iteration getestet wird, sollten wir diese Codesegmente sorgfältig überprüfen, denn oft gibt es eine andere Lösung für diese Spezialfälle, mit der diese Tests vermieden werden können. Eigentlich möchten wir im Rumpf einer Schleife nur Anweisungen haben, die bei jedem Durchlauf ausgeführt werden, und Spezialfälle wollen wir so weit wie möglich vermeiden. Es wird nicht immer möglich sein, alle Ausnahmefälle zu beseitigen, aber diejenigen, die nur bei dem ersten oder letzten Schleifendurchlauf angewendet werden, werden vermutlich am ehesten zu vermeiden sein.

Sie haben sicher auch bemerkt, dass im Laufe des Iterationsprozesses die Zellen im „Zentrum“ in der nächsten Runde immer zum „linken“ Nachbarn werden; und der „rechte“ Nachbar wird zur nächsten Zelle im „Zentrum“. Dadurch eröffnet sich die Möglichkeit, die nächsten Werte von **links** und **zentrum** am Ende des Schleifenrumpfs so einzurichten, dass sie schon für den nächsten Durchlauf vorbereitet sind; das heißt:

```
naechsterZustand[i] = (links + zentrum + rechts) % 2;
links = zentrum;
zentrum = rechts;
```

Damit muss nur noch **rechts** explizit vom Array **zustand** in der nächsten Iteration eingerichtet werden. Interessanterweise liefert diese Variante auch einen Weg, das Problem zu lösen, wie die Zuweisung an **links** im der ersten Schleifendurchlauf zu behandeln ist. Die obige Änderung bedeutet: wird der Schleifenrumpf – mit Ausnahme der ersten Iteration – betreten, dann sind **links** und **zentrum** bereits vom vorherigen Schleifendurchlauf auf die Werte gesetzt worden, die sie zur Berechnung des nächsten Zustands haben sollten. Wir müssen also nur sicherstellen, dass **links** für die nicht vorhandene Zelle schon auf 0 gesetzt und **zentrum** unmittelbar vor dem ersten Betreten der Schleife auf **zustand[0]** gesetzt wurde:

```
int links = 0;
int zentrum = zustand[0];
for(int i = 0; i < zustand.length; i++) {
    int rechts = i + 1 < zustand.length ? zustand[i+1] : 0;
    naechsterZustand[i] = (links + zentrum + rechts) % 2;
    links = zentrum;
    zentrum = rechts;
}
```

Beachten Sie, dass die Variablendeklarationen für **links** und **zentrum** nun nicht mehr innerhalb der Schleife stehen, sodass deren Werte zwischen den Durchläufen erhalten bleiben.

In der nächsten Übung werden Sie aufgefordert, diese Änderungen in Ihrer Version des Projekts zu implementieren. Eine Version, die all diese Änderungen bereits erhält, finden Sie im Projekt *Automat-v2*, falls Sie Schwierigkeiten haben oder Ihre Lösung überprüfen möchten; versuchen Sie jedoch unbedingt zuerst, die Aufgaben selbst durchzuführen, bevor Sie nachsehen.

Übung 7.31 Schreiben Sie die Methode **aktualisieren** so um, wie wir es oben besprochen haben.

Übung 7.32 Fügen Sie eine Methode **berechneNaechstenZustand** zur Klasse **Automat** hinzu, die drei Werte – **links**, **zentrum** und **rechts** – entgegennimmt und die Berechnung des Werts für den nächsten Zustand zurückgibt. Der nächste Zustand einer Zelle wurde bisher mit der folgenden Codezeile berechnet:

```
naechsterZustand[i] = (links + zentrum + rechts) % 2;
```

Verändern Sie diese Zeile, um stattdessen Ihre neue Methode zu verwenden.

Übung 7.33 Experimentieren Sie mit unterschiedlichen Wegen, den nächsten Zustand einer Zelle zu berechnen, indem die Werte von **links**, **zentrum** und **rechts** kombiniert werden. Stellen Sie immer sicher, dass Sie das Ergebnis noch einer Modulo-2-Operation unterziehen. Sie müssen nicht alle drei Werte in die Berechnung einbeziehen. Hier sind ein paar Möglichkeiten, die Sie vielleicht ausprobieren möchten:

$$(\text{links} + \text{rechts}) \% 2$$

$$(\text{zentrum} + \text{rechts} + \text{zentrum} * \text{rechts} + \text{links} * \text{zentrum} * \text{rechts}) \% 2$$

Wie viele unterschiedliche Mengen eindeutiger Regeln gibt es Ihrer Meinung nach, um den nächsten 0- oder 1-Zustand einer Zelle zu berechnen, ausgehend von den drei binären Werten in **links**, **zentrum** und **rechts**? Gibt es unendlich viele Möglichkeiten?

Die letzte Verbesserung, die wir noch am Automaten machen möchten, ist, den Test auf das Ende des Arrays zu vermeiden, bevor wir eine Zuweisung an **rechts** vornehmen. Dafür gibt es eine häufig genutzte Lösung – das Array um ein Element erweitern! Diese Lösung hört sich zunächst ein bisschen nach Mogeln an, doch die Erweiterung eines Arrays um ein einzelnes Element kann manchmal zu klarerem Code führen, wenn es gut durchdacht und richtig eingesetzt wird. Die Idee hier ist, dass das zusätzliche Element den Wert 0 enthält, um als rechter Nachbar der letzten Zelle zu dienen. Dieser Nachbar wird selbst jedoch nicht als Zelle in den Automaten integriert. Er wird niemals geändert. Die überarbeitete Version des Rumpfs von **aktualisieren** lautet:

```
int links = 0;
int zentrum = zustand[0];
for(int i = 0; i < anzahlZellen; i++) {
    int rechts = zustand[i + 1];
    naechsterZustand[i] = berechneNaechstenZustand(links, zentrum, rechts);
    links = zentrum;
    zentrum = rechts;
}
```

Beachten Sie bitte, dass die Schleifenbedingung verändert wurde, weil die Indexvariable nicht mehr mit der Länge des (erweiterten) Arrays verglichen werden sollte, sondern mit der tatsächlichen Anzahl der Zellen im Automaten. Diese Änderung ist wichtig, um sicherzustellen, dass **[i + 1]** niemals über die Grenzen des Arrays hinausgeht, wenn der Wert von **rechts** gesetzt wird.

Eine Projektversion einschließlich dieses Codes können Sie auch im Projekt *Automat-v3* finden.

Übung 7.34 Führen Sie diese Verbesserungen in Ihrem eigenen Code durch.

7.5.3 Zuordnungstabellen

In Kapitel 6 haben wir die Klasse `HashMap` besprochen, die es ermöglicht, Verbindungen zwischen Objekten in der Form eines *Schlüssel-Werte-Paars* zu erzeugen. Eine Map ist eine allgemeine Form einer *Zuordnungstabelle*: der Schlüssel wird verwendet, um einen ihr zugeordneten Wert in einer Tabelle, der Map, zu finden. Arrays sind häufig eine praktische und effiziente Möglichkeit, um spezielle Zuordnungstabellen zu implementieren, bei denen der Schlüssel kein Objekt, sondern ein Wert aus einem ganzzahligen Wertebereich ist. Dieser Bereich sollte als kleinsten Wert 0 haben, damit es keine Konflikte mit der Indexierungsart für Sammlungen gibt. Natürlich könnte ein Bereich, dessen kleinster Wert ungleich 0 ist, leicht in einen 0-basierten Bereich umgewandelt werden, indem eine positive oder negative Zahl (Offset) zu allen Werten addiert wird. Solange die Größe des Wertebereichs bekannt ist, kann ein Array fester Größe erzeugt werden, um die Werte der (Schlüssel-Wert)-Paare aufzunehmen, wobei der Schlüssel ein 0-basierter ganzzahliger Index ist.

Das Konzept einer Zuordnungstabelle wollen wir mithilfe des zellulären Automaten illustrieren. Wenn Sie mit verschiedenen Formeln für die Berechnung des nächsten Zustands einer Zelle experimentiert haben, dann ist Ihnen vielleicht schon aufgefallen, dass der eindimensionale Automat, den wir implementiert haben, nicht wirklich unendlich viele Möglichkeiten hat, den nächsten Zustand einer Zelle zu bestimmen. Diese Begrenzung resultiert aus den folgenden beiden Eigenschaften:

- Es gibt nur acht mögliche unterschiedliche Kombinationen von linken, zentralen und rechten Werten von 1 und 0: 000, 001, 010, ..., 111.
- Für jede der acht Kombinationen kann das Ergebnis nur eine von zwei Möglichkeiten sein: 0 oder 1.

Zusammengenommen bedeuten diese Eigenschaften, dass es tatsächlich nur 256 (d.h. 2^8) Möglichkeiten gibt, wie sich der Automat verhalten kann. In der Tat ist die Anzahl der eindeutigen Verhaltensmöglichkeiten noch beträchtlich kleiner – 88 gesamt.

Wolfram-Codes

1983 hat Stephen Wolfram eine Studie über alle möglichen 256 elementaren zellulären Automaten veröffentlicht. Er stellte ein numerisches System vor, mit dem das Verhalten jedes Automatentyps definiert werden kann, und der jedem Automaten zugeordnete Code ist als *Wolfram-Code* bekannt. Ist der numerische Code gegeben, so kann man sehr leicht die anwendbare Regel für Zustandsänderungen herausfinden, wenn die Werte einer Zelle und seiner beiden Nachbarn vorliegen, weil die Regel im Code selbst verschlüsselt ist. Um zu sehen, wie dies in der Praxis funktioniert, empfehlen wir Ihnen **Übung 7.36** oder führen Sie eine Websuche nach „elementarer zellulärer Automat“ durch.

Diese relativ bescheidene Anzahl an Möglichkeiten erlaubt es, die Determinierung des nächsten Zustands einer Zelle zu implementieren, und zwar über eine Zuordnungstabelle anstelle einer Funktionsauswertung. Zwei Dinge sind dazu nötig:

- Eine Möglichkeit, ein (links, zentrum, rechts)-Triplet in eine Zahl aus dem Bereich 0–7 umzuwandeln. Diese Werte werden als Indexschlüssel der Zuordnungstabelle verwendet.
- Ein Integer-Array mit 8 Elementen, das den Wert für den nächsten Zustand für die acht Triplet-Kombinationen speichert. Dies wird die Tabelle sein.

Der folgende Ausdruck zeigt, wie wir die Dreiergruppe in eine ganze Zahl aus dem Bereich 0–7 umwandeln können:

```
links * 4 + zentrum * 2 + rechts
```

Und eine Zustandstabelle kann wie folgt eingerichtet werden:

```
int[] zustandstabelle = new int[] {
    0, 1, 0, 0, 1, 0, 0, 1,
};
```

Dies illustriert den Einsatz eines *Array-Initialisierers* – eine Liste der Werte, die in einem neu erzeugten Array gespeichert werden sollen, in geschweiften Klammern eingefasst. Beachten Sie, dass die Größe des erzeugten Arrays nicht in den eckigen Klammern von `new int[]` angegeben werden muss, weil der Compiler zählen kann, wie viele Elemente in dem Initialisierer sind, daher kann er das Array-Objekt in exakt dieser Größe erzeugen. Beachten Sie auch das abschließende Komma nach dem letzten Wert im Initialisierer; dieses Komma ist optional. Eine Implementierung dieser Version finden Sie in *Automat-v4*.

Übung 7.35 Experimentieren Sie mit unterschiedlichen Mustern der Zuordnungstabelle in *Automat-v4*.

Übung 7.36 Zusatzaufgabe. In dieser Übung geht es darum, die Zuordnungstabelle einzurichten, die auf einem zusätzlichen Integer-Parameter basiert, der dem Konstruktor von **Automat** übergeben wird. Die acht 1/0-Werte in der Zuordnungstabelle könnten als acht Binärziffern interpretiert werden, die einen numerischen Wert aus dem Bereich 0–255 verschlüsseln, ganz ähnlich wie wir das Triplet von Zellzuständen als eine ganze Zahl im Bereich von 0–7 interpretiert haben. Das niedrigstwertige Bit wäre der Wert in **zustandstabelle[0]** und das höchstwertige Bit in **zustandstabelle[7]**. Damit wäre das Muster, das wir oben bei der Initialisierung von **zustandstabelle** verwendet haben, eine Verschlüsselung des Dezimalwerts 146 (128 + 16 + 2). Und tatsächlich werden die 256 Wolfram-Codes genau so verwendet – ein Code aus dem Bereich 0–255 wird in seine binäre 8-Bit-Binärdarstellung verwandelt und die Binärziffern werden zum Einrichten der Zustandstabelle benutzt. Fügen Sie einen Integer-Parameter zum Konstruktor von **Automat** hinzu, der einen Wolfram-Code enthält, und benutzen Sie diesen, um **zustandstabelle** zu initialisieren. Dazu müssen Sie herausfinden, wie die einzelnen Binärziffern aus einer ganzen Zahl gewonnen werden. Verwenden Sie das niedrigstwertige Bit, um Element 0 in der Zuordnungstabelle einzurichten, das nächste Bit, um Element 1 einzurichten, und so weiter. *Hinweis:* Sie könnten dies entweder mithilfe von Integer-Operationen wie `%` oder `/` tun oder mit Bit-Manipulationsoperatoren wie `>>` und `&`.

7.6 Arrays mit mehr als einer Dimension (fortgeschritten)

Dieser Abschnitt gehört zu den Teilen dieses Buchs mit fortgeschrittenen Konzepten. Sie können ihn im Moment gefahrlos überspringen, ohne irgendetwas zu verpassen, was Sie für spätere Kapitel benötigen würden. Wir werden zweidimensionale Arrays besprechen, eine recht spezielle, aber nützliche Datenstruktur. Wenn Sie das oben diskutierte *Automat*-Projekt interessiert hat, wird Ihnen diese reichere Variante Spaß machen.

Arrays mit mehr als einer Dimension sind einfach eine Erweiterung des eindimensionalen Konzepts. Zum Beispiel ist ein zweidimensionales Array ein eindimensionales Array von eindimensionalen Arrays. Es hat sich durchgesetzt, dass die erste Dimension eines zweidimensionalen Array die Reihen eines Gitters repräsentiert und die zweite Dimension die Spalten. Die Tatsache, dass zweidimensionale Arrays eigentlich Arrays von Arrays sind, bedeutet, dass zweidimensionale Arrays nicht unbedingt rechteckig sein müssen: Jede Reihe könnte eine andere Anzahl an Elementen besitzen. Diese besondere Form werden wir allerdings nicht weiter untersuchen. In diesem Abschnitt begrenzen wir unsere Diskussion der mehrdimensionalen Arrays auf den zweidimensionalen Fall, da weitere Dimensionen auf dieselbe Art zustande kommen.

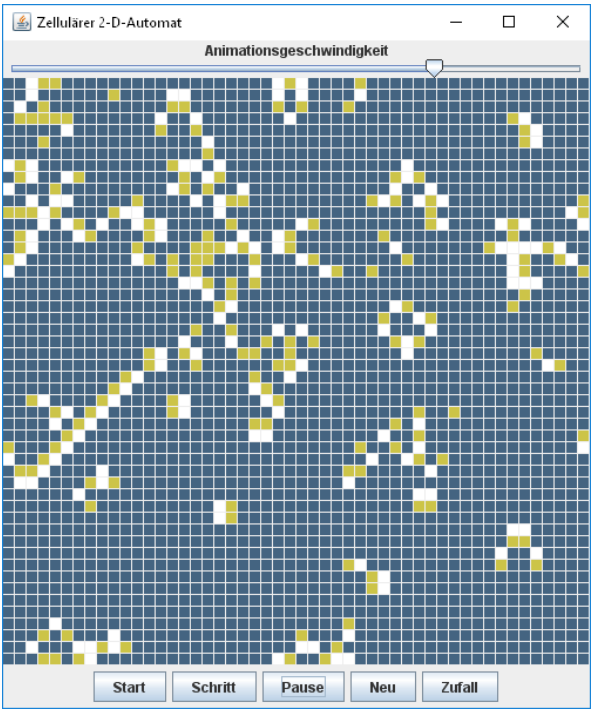
7.6.1 Das Projekt *Brain*

Die Regeln, die wir beim eindimensionalen Automaten kennengelernt haben, lassen sich auf vielerlei Weisen auf zwei Dimensionen erweitern. Das vermutlich berühmteste Beispiel eines zweidimensionalen Automaten ist *Conways Spiel des Lebens*, bei dem die Zellzustände auch wieder binär sind (also entweder 1 oder 0). Wir werden einen leicht anspruchsvolleren zellulären Automaten implementieren, der unter dem Namen *Brian's Brain* bekannt ist, weil er von Brian Silverman erfunden wurde. Hierbei werden wir eine Klasse verwenden, die die Zellen repräsentiert, weil das Verhalten und die Interaktionen erheblich komplexer als bei denen des elementaren Automaten sind, den wir uns im vorigen Abschnitt angesehen haben.

Übung 7.37 Öffnen Sie das Projekt *Brain*, erzeugen Sie ein **Umgebung**-Objekt und verwenden Sie die GUI-Steuerelemente, um eine zufällige Anfangseinstellung für den Automaten zu erzeugen. Führen Sie dann den Automaten entweder schrittweise aus oder mithilfe der Start-/Pause-Buttons, um ein Gefühl dafür zu bekommen, wie sich der Automat verhält (Abbildung 7.3).

Abbildung 7.3

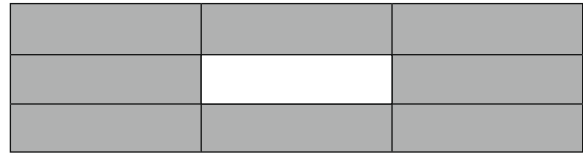
Ein zweidimensionaler Automat aus dem *Brain*-Projekt.



Die Zellen in *Brian’s Brain* befinden sich immer in einem von drei möglichen Zuständen, die als *lebend*, *sterbend* oder *tot* (*alive*, *dying*, *dead*) bezeichnet werden. Diese Zustände werden in unserem Projekt durch verschiedene Farben dargestellt. Wir möchten jedoch noch einmal deutlich machen, dass dies einfach nur übliche Bezeichnungen sind, die keine objektive Bedeutung haben. Wir werden die willkürlich gewählten Werte 0, 1 und 2 verwenden, um diese Zustände dazustellen, obwohl wir – anders als beim eindimensionalen Automaten – keinen Ausdruck damit berechnen werden. Der nächste Zustand einer Zelle wird auf der Basis seines eigenen Zustands und der Zustände seiner Nachbarn berechnet. Bei einem zweidimensionalen Automaten besteht die Nachbarschaft in der Regel aus den acht direkten Nachbarn der Zelle. Diese sind unter dem Namen *Moore-Nachbarschaft* bekannt (Abbildung 7.4).

Abbildung 7.4

Die äußeren grauen Zellen stellen die Moore-Nachbarschaft der Zelle im Zentrum dar.



- Die Regeln zum Aktualisieren des Zellzustands sind die folgenden:
- Eine Zelle, die *lebend* ist, verändert ihren Zustand zu *sterbend*.
 - Eine Zelle, die *sterbend* ist, verändert ihren Zustand zu *tot*.
 - Eine Zelle, die *tot* ist, verändert ihren Zustand zu *lebend*, wenn genau zwei ihrer Nachbarn *lebend* sind, andernfalls bleibt sie *tot*.

Listing 7.3 zeigt die Klasse `Zelle`, die diese Konzepte implementiert.

```

import java.util.*;

/**
 * Eine Zelle in einem zweidimensionalen zellulären Automaten.
 * Die Zelle hat mehrere mögliche Zustände.
 * Dies ist eine Implementierung der Regeln von "Brian's Brain".
 * @see https://en.wikipedia.org/wiki/Brian%27s_Brain
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class Zelle
{
    // Die möglichen Zustände.
    public static final int LEBEND = 0, TOT = 1, STERBEND = 2;
    // Die Anzahl der möglichen Zustände.
    public static final int ANZ_ZUSTAENDE = 3;

    // Der Zustand der Zelle.
    private int zustand;
    // Die Nachbarn der Zelle.
    private Zelle[] nachbarn;

    /**
     * Setze den Anfangszustand.
     * @param anfangszustand der Anfangszustand
     */
    public Zelle(int anfangszustand)
    {
        zustand = anfangszustand;
        nachbarn = new Zelle[0];
    }

    /**
     * Lege den nächsten Zustand dieser Zelle fest, basierend auf dem
     * dem Zustand der Nachbarn.
     * Dies ist eine Implementierung der Regeln für "Brian's Brain".
     * @return den nächsten Zustand
     */
    public int gibNaechstenZustand()
    {
        if(zustand == TOT) {
            // Zähle die Anzahl der Nachbarn, die LEBEND sind.
            int lebendZaehler = 0;
            for(Zelle n : nachbarn) {
                if(n.gibZustand() == LEBEND) {
                    lebendZaehler++;
                }
            }
            return lebendZaehler == 2 ? LEBEND : TOT;
        }
        else if(zustand == STERBEND) {
            return TOT;
        }
        else {
            return STERBEND;
        }
    }

    /**
     * Erhalte die Liste der benachbarten Zellen und erstelle eine Kopie.
     * @param nachbarnListe die benachbarten Zellen
     */
    public void setzeNachbarn(ArrayList<Zelle> nachbarnListe)
    {
        nachbarn = new Zelle[nachbarnListe.size()];
        nachbarnListe.toArray(nachbarn);
    }

    // weitere Methoden hier ausgelassen
}

```

Listing 7.3

Eine Zelle eines zweidimensionalen Automaten.

Jede Zelle verwaltet eine Liste seiner Nachbarn in einem Array. Das Array wird nach einem Aufruf der Methode **setzeNachbarn** eingerichtet. Diese Methode erhält eine Liste von Zellen und kopiert diese Liste in ein Array-Objekt mithilfe der **toArray**-Methode der Liste.

7.6.2 Einrichten des Arrays

Listing 7.4 zeigt einige dieser Elemente aus der Klasse **Umgebung**, die das zweidimensionale Array der **Zelle**-Objekte einrichtet, um den Automaten zu modellieren.

Listing 7.4

Die Klasse **Umgebung**.

```
public class Umgebung
{
    // Das Zellengitter.
    private Zelle[][] zellen;
    // Visualisierung der Umgebung.
    private final Umgebungsansicht ansicht;

    /**
     * Erzeuge eine Umgebung der angegebenen Größe.
     * @param anzahlZeilen die Anzahl der Zeilen
     * @param anzahlSpalten die Anzahl der Spalten
     */
    public Umgebung(int anzahlZeilen, int anzahlSpalten)
    {
        setup(anzahlZeilen, anzahlSpalten);
        zufallsaufbau();
        ansicht = new Umgebungsansicht(this, anzahlZeilen, anzahlSpalten);
        ansicht.zeigeZellen();
    }

    // weitere Methoden hier ausgelassen

    /**
     * Richte eine neue Umgebung der angegebenen Größe ein.
     * @param anzahlZeilen die Anzahl der Zeilen
     * @param anzahlSpalten die Anzahl der Spalten
     */
    private void setup(int anzahlZeilen, int anzahlSpalten)
    {
        zellen = new Zelle[anzahlZeilen][anzahlSpalten];
        for(int zeile = 0; zeile < anzahlZeilen; zeile++) {
            for (int spalte = 0; spalte < anzahlSpalten; spalte++) {
                zellen[zeile][spalte] = new Zelle();
            }
        }
        setupNachbarn();
    }
}
```

Die Syntax zum Deklarieren einer Array-Variable mit mehr als einer Dimension ist eine Erweiterung des eindimensionalen Falls – ein Paar von leeren eckigen Klammern für jede Dimension:

```
Zelle[][] zellen;
```

Ähnlich legt die Erzeugung des Array-Objekts mithilfe von `new` die Länge jeder Dimension fest:

```
zellen = new Zelle[anzahlZeilen][anzahlSpalten];
```

Denken Sie daran, dass zu diesem Zeitpunkt nicht tatsächlich **Zelle**-Objekte erzeugt werden – sondern nur das Array-Objekt, das in der Lage sein wird, die **Zelle**-Objekte zu speichern, sobald diese erzeugt wurden.

Die Methode **setup** verwendet ein typisches Codemuster, um über alle Elemente eines zweidimensionalen Arrays zu iterieren: eine geschachtelte **for**-Schleife. Das Muster sieht ungefähr so aus:

```
for(int zeile = 0; zeile < anzahlZeilen; zeile++) {
    for(int spalte = 0; spalte < anzahlSpalten; spalte++) {
        // ...
    }
}
```

In dieser Schleife wird dann jede Zelle eingerichtet, indem das Zelle-Objekt erzeugt und der aktuellen Position im Array zugewiesen wird:

```
zellen[zeile][spalte] = new Zelle();
```

Schließlich ruft die Methode **setup** eine separate Methode **setupNachbarn** auf, um jeder **Zelle** ihre Moore-Nachbarschaft von benachbarten Zellen zu übergeben.

Sie werden sich aus der Diskussion des eindimensionalen Arrays erinnern, dass wir am linken und rechten Ende des Automaten aufpassen mussten, damit auch die Randzellen die volle Anzahl an Nachbarn bekamen. Dasselbe gilt nun auch beim zweidimensionalen Automaten. Zum Beispiel hat die **Zelle** an Position **zellen[0][0]** nur drei Nachbarn anstelle von acht. Ein möglicher Ansatz wäre es, das zweidimensionale Array um jeweils eine zusätzliche Zeile oben und unten sowie eine zusätzliche Spalte links und rechts zu erweitern – wodurch ein **(anzahlZeilen+2) (anzahlSpalten+2)**-Automat entsteht – und alle Zellen in diesen zusätzlichen Position als *tot* anzunehmen. Doch wir ziehen einen „Rundum“-Ansatz zur Berechnung der Nachbarschaft vor – mit anderen Worten, Zellen in der obersten Zeile haben als Nachbarn die Zellen aus der untersten Reihe und Zellen in der ganz linken Spalte haben als Nachbarn die Zellen in der ersten rechten Spalte (und jeweils umgekehrt). Dies wird auch *ringförmige* oder *toroidale Anordnung* genannt. Auf diese Weise besitzt jede Zelle im Automaten die vollständige Moore-Nachbarschaft von acht Zellen. Jetzt müssen wir noch einen Weg finden, die Fälle entlang der Ränder zu behandeln, ohne auf einen Code für Spezialfälle zurückzugreifen. Wenn wir die Ränder im Moment einmal noch ignorieren, kann man die Nachbarn der Zelle an Position **[zeile][spalte]** an allen Positionen mit Abstand **+1** und **-1** von **zeile** und **spalte** finden; z.B. **[zeile+1][spalte]**, **[zeile+1][spalte+1]**, **[zeile-1][spalte]** und so weiter. Um eine Liste der Nachbarn zu erstellen, könnten wir also Code nach folgendem Muster schreiben:

```
Zelle zelle = zellen[zeile][spalte];
for(int dz = -1; dz <= 1; dz++) {
    for(int ds = -1; ds <= 1; ds++) {
        füge den Nachbarn an zellen[zeile+dz][spalte+ds] hinzu;
    }
}
```

Beachten Sie, dass wenn sowohl **dz** als auch **ds** null sind, dann ist der „Nachbar“ in Wirklichkeit **zelle** selbst und sollte nicht in die Liste der Nachbarn aufgenommen werden. Dies müssen wir für unsere Lösung also noch im Hinterkopf behalten.

Nun müssen wir sicherstellen, dass wir den Wert **0** erhalten, wenn **zeile + 1 == anzahlZeilen** ist, und wir **(anzahlZeilen-1)** erhalten, wenn **zeile + -1 == -1** ist (dasselbe gilt für **spalte**). Die Lösung der ersten Anforderung ist relativ einfach – der Modulo-Operator liefert genau das. Doch damit sind nicht beide Anforderungen abgedeckt: **(-1 % anzahlZeilen)** ergibt **-1** anstatt **(anzahlZeilen - 1)**. Hier können wir einen kleinen Trick anwenden, der auf der Tatsache beruht, dass **((x + anzahlZeilen) % anzahlZeilen) == (x % anzahlZeilen)**, wenn **x** nicht negativ ist, was bedeutet, dass **((zeile + 1 + anzahlZeilen) % anzahlZeilen)** zum selben Ergebnis führt wie **((zeile + 1) % anzahlZeilen)**. Ist **x** jedoch negativ, funktioniert dies nicht, und wenn wir **anzahlZeilen** hinzufügen, bevor wir den Modulo-Operator anwenden, dann ergibt **((-1 + anzahlZeilen) % anzahlZeilen)** wunschgemäß **(anzahlZeilen - 1)**. Der Ausdruck **((zeile + dz + anzahlZeilen) % anzahlZeilen)** liefert also den Index, den wir sowohl in positiven wie in negativen Fällen benötigen, um die Nachbarschaft einer Zelle in einem umlaufenden Gitter festzulegen.

Listing 7.5 zeigt die vollständige Version der Methode `setupNachbarn` aus der `Umgebung`-Klasse, die diese Technik einsetzt. Beachten Sie, dass eine Zelle aus ihrer eigenen Nachbarschaftsliste entfernt wird, bevor die Liste der Zelle übergeben wird – womit der Fall, dass `dz` und `ds` beide null sind, behandelt wird.

Listing 7.5
Einrichten der
Nachbarzellen.

```
/**
 * Gib jeder Zelle eine Liste ihrer Nachbarn.
 */
private void setupNachbarn()
{
    int anzahlZeilen = zellen.length;
    int anzahlSpalten = zellen[0].length;
    // 8 Nachbarn plus die Zelle selbst
    ArrayList<Zelle> nachbarn = new ArrayList<>(9);
    for(int zeile = 0; zeile < anzahlZeilen; zeile++) {
        for(int spalte = 0; spalte < anzahlSpalten; spalte++) {
            Zelle zelle = zellen[zeile][spalte];
            // Dieser Prozess wird auch für die Zelle selbst ausgeführt.
            for(int dz = -1; dz <= 1; dz++) {
                for(int ds = -1; ds <= 1; ds++) {
                    int nz = (anzahlZeilen + zeile + dz) % anzahlZeilen;
                    int ns = (anzahlSpalten + spalte + ds) % anzahlSpalten;
                    nachbarn.add(zellen[nz][ns]);
                }
            }
            // Die Menge der Nachbarn sollte die Zelle an Position (zeile,spalte)
            // nicht enthalten, deshalb wird sie entfernt.
            nachbarn.remove(zelle);
            zelle.setzeNachbarn(nachbarn);
            nachbarn.clear();
        }
    }
}
```

Übung 7.38 *Conways Spiel des Lebens* ist ein deutlich einfacherer Automat als *Brian's Brain*. Beim *Spiel des Lebens* besitzt eine Zelle nur zwei Zustände: lebend und tot. Der nächste Zustand einer Zelle wird folgendermaßen bestimmt:

- Es werden alle Nachbarn der Zelle gezählt, die *lebend* sind.
- Ist die Zelle *tot* und hat sie genau drei lebende Nachbarn, dann ist ihr Folgezustand *lebend*, ansonsten ist der nächste Zustand *tot*.
- Ist die Zelle *lebend*, dann hängt ihr nächster Zustand auch von der Anzahl der lebenden Nachbarn ab: Sind es weniger als zwei oder mehr als drei, dann ist der Folgezustand *tot*, andernfalls *lebend*.

Speichern Sie eine Kopie des *Brain*-Projekts unter dem Namen *Spiel-des-Lebens* und verändern Sie die Klasse `Zelle` so, dass diese die Regeln von *Spiel des Lebens* implementiert. Es sollte nicht nötig sein, irgendwelche anderen Klassen zu verändern.

Übung 7.39 Bearbeiten Sie die Klasse `Umgebung` in Ihrer *Spiel-des-Lebens*-Version, sodass die Umgebung nicht mehr ringförmig ist. Mit anderen Worten, der nächste Zustand einer Zelle an einer äußeren Reihe oder Spalte der Umgebung wird bestimmt, indem angenommen wird, dass „Nachbarn“ jenseits der äußeren Grenzen der Umgebung immer *tot* sind. Stellen Sie einen großen Unterschied zwischen dem Verhalten dieser Version und der ringförmigen Variante fest?

Übung 7.40 Verwenden Sie wahlweise entweder eine Kopie des *Brain*-Projekts oder Ihres *Spiel-des-Lebens*-Projekts, um ein neues Projekt zu implementieren, bei dem die Zellen die Zustände *lebend* oder *tot* annehmen können. Lebende Zellen sollten den Eindruck erwecken, als „wanderten“ sie durch die

zweidimensionale Umgebung. Weisen Sie den Zellen eine Bewegungsrichtung zu. Eine lebende Zelle sollte sich bei jedem Schritt in die zugeordnete Richtung „bewegen“ – mit anderen Worten sollte eine lebende Zelle einen ihrer Nachbarn dazu bringen, im nächsten Schritt *lebend* zu werden, während die Zelle selbst *tot* wird – dies ergibt die Illusion der Bewegung. Wenn eine lebende Zelle am Rand der Umgebung versuchen würde, sich über den Rand hinaus zu „bewegen“, sollte sie stattdessen ihre Richtung zufällig ändern und im nächsten Schritt noch *lebend* bleiben; d.h., sie wird in diesem Fall nicht den Zustand eines Nachbarn ändern.

Überprüfen Sie Ihre Implementierung dieser Regeln, indem Sie anfangs nur eine einzige lebende Zelle in die Umgebung setzen. Haben Sie mehrere lebende Zellen, dann müssen Sie sich überlegen, was passieren soll, wenn eine Zelle versucht, sich auf eine Position zu „bewegen“, die gerade erst von einer anderen Zelle verlassen wurde, oder wenn zwei Zellen versuchen, dieselbe Nachbarzelle zu besetzen.

7.7 Arrays und Streams (fortgeschritten)

Die Prinzipien zur Stream-Verarbeitung, die wir in Kapitel 5 eingeführt haben, können auf Arrays ebenso gut wie auf flexible Sammlungen angewandt werden. Man erhält einen Stream aus den Inhalten eines Arrays, indem das Array an die statische Methode `stream` von `java.util.Arrays` übergeben wird. Wenn das Array vom Typ `int[]` ist, dann wird ein Objekt vom Typ `java.util.Stream.IntStream` zurückgeliefert:

```
int[] zahlen = { 1, 2, 3, 4, 5 }
IntStream stream = Arrays.stream(zahlen);
```

Es gibt außerdem noch `DoubleStream`- und `LongStream`-Typen für die entsprechenden Stream-Versionen von `double[]`- und `long[]`-Arrays. In den anderen Fällen wird für ein Array mit Typ `T` ein Stream vom Typ `Stream<T>` erzeugt.

Die Standardoperationen auf Streams wie `filter`, `limit`, `map`, `reduce`, `skip` und so weiter stehen natürlich alle für den resultierenden Stream zur Verfügung, sodass die Daten in einer Sammlung fester Größe auf genau dieselbe Weise wie in den flexiblen Sammlungen verarbeitet werden können, sobald sie in einen Stream umgewandelt wurden.

Die statische Methode `range` der Klasse `IntStream` wird häufig verwendet, um einen Stream aus den Integer-Indizes einer Sammlung zu erzeugen; zum Beispiel:

```
IntStream indices = IntStream.range(0, zahlen.length);
```

Der Wert von `zahlen.length` wird nicht in den Stream aufgenommen. Das folgende Beispiel zeigt, wie man die Indizes von Werten eines Arrays identifizieren könnten, die über einem bestimmten Schwellenwert liegen. Beachten Sie, dass wir in diesem speziellen Fall an den Indizes und nicht an den Werten interessiert sind:

```
int[] ueber = IntStream.range(0, stundenzaehler.length)
    .filter(i -> stundenzaehler[i] > schwellwert).toArray();
```

Die terminale `toArray`-Methode des Streams gibt ein Array zurück, welches die Indexwerte enthält, die nach dem Filtern zurückbleiben.



Zusammenfassung

In diesem Kapitel haben wir Arrays als Möglichkeit diskutiert, um Sammlungsobjekte fester Größe zu erzeugen. Eindimensionale Arrays sind ein Sammlungstyp, der gut geeignet für solche Situationen ist, in denen die Anzahl der zu speichernden Elemente festgelegt und im Voraus bekannt ist. Sie sind auch syntaktisch ein wenig leichter als Sammlungen flexibler Größe wie `ArrayList`, insbesondere wenn sie zur Speicherung von Werten der primitiven Typen verwendet werden. Abgesehen von diesen speziellen Anwendungen bieten sie jedoch keine Funktionen, die nicht auch bereits mit Sammlungen wie Listen, Maps und Sets verfügbar wären.

NEUE BEGRIFFE IN DIESEM KAPITEL

Array, **for**-Schleife, Konditionaloperator, Zuordnungstabelle



Zusammenfassung der Konzepte

- **Array** Ein Array ist eine besondere Art von Sammlung, die eine festgelegte Anzahl von Elementen halten kann.
- **for-Schleife** Eine iterative Steuerungsstruktur, die häufig verwendet wird, wenn eine Indexvariable benötigt wird, um Folgeelemente aus einer Sammlung auszuwählen, wie eine `ArrayList` oder ein Array.

Übung 7.41 Schreiben Sie den folgenden Code neu, indem Sie die statische Methode `arraycopy` aus der Klasse `System` verwenden:

```
int[] kopie = new int[original.length];
for(int i = 0; i < original.length; i++) {
    kopie[i] = original[i];
}
```

Übung 7.42 Beschreiben Sie, was die folgenden statischen Methoden der Klasse `java.util.Arrays` machen: `asList`, `binarySearch`, `fill` und `sort`. Wenn es mehrere Versionen einer Methode gibt, verwenden Sie eine, die auf einem Integer-Array arbeitet.

Übung 7.43 Versuchen Sie, ein paar Beispielcodesegmente zu schreiben, mit dem Sie jede der Methoden aus der vorigen Übung benutzen können.

Übung 7.44 Schreiben Sie Anweisungen, um alle Elemente eines zweidimensionalen Arrays, `original`, in ein neues Array, `kopie`, zu kopieren. Sie sollten auch Anweisungen zur Erzeugung des `kopie`-Arrays schreiben.

Übung 7.45 Untersuchen Sie einige andere zweidimensionale zelluläre Automaten und implementieren Sie deren Regeln im *Brain*-Projekt.



KAPITEL

8

Klassenentwurf

Lernziele

Zentrale Konzepte in diesem Kapitel: Entwurf nach Zuständigkeiten, Kohäsion, Kopplung, Refactoring

Java-Konstrukte in diesem Kapitel: Aufzählungstypen, `switch`

In diesem Kapitel untersuchen wir die Faktoren, die den Entwurf einer Klasse beeinflussen. Wann ist ein Klassenentwurf gut, wann ist er schlecht? Ein guter Klassenentwurf kann kurzfristig mehr Aufwand bedeuten als ein schlechter, aber auf lange Sicht zahlt sich dieser Mehraufwand fast immer aus. Um gute Klassen zu schreiben, kann man sich an einige Entwurfsprinzipien halten. Insbesondere stellen wir hier die Sichtweise vor, dass ein Klassenentwurf sich an Zuständigkeiten orientieren sollte und dass Klassen ihre Daten kapseln sollten.

Dieses Kapitel orientiert sich, wie die vorherigen Kapitel auch, an einem Projekt. Sie können es einfach nur lesen und seiner Argumentation folgen, oder Sie können es sehr viel intensiver bearbeiten, indem Sie die Übungen am Projekt parallel zum Text bearbeiten.

Die Projektarbeit ist in drei Teile unterteilt. Im ersten Teil diskutieren wir einige notwendige Änderungen am Quelltext und entwickeln und zeigen komplette Lösungen zu den Übungen. Die Gesamtlösung für diesen Teil ist auch als ein eigenes Projekt im Begleitmaterial zu finden. Der zweite Teil schlägt zusätzliche Änderungen und Erweiterungen vor, die auf der Ebene des Klassenentwurfs diskutiert werden; die konkrete Umsetzung in der Programmierung überlassen wir jedoch dem Leser.

Der dritte Teil schlägt dann noch weitergehende Verbesserungen in Form von Übungen vor. Hier liefern wir keine Lösungen – die Übungen wenden das vorgestellte Material dieses Kapitels an.

Die Implementierung aller Teile ist ein gutes Programmierprojekt, das über mehrere Wochen laufen kann. Sie kann auch sehr gut in Gruppenarbeit vorgenommen werden.